

Programación para Física y Astronomía

Departamento de Física.

Coordinadora: C Loyola

Profesores C Femenías / D Basantes

Primer Semestre 2025

Universidad Andrés Bello

Departamento de Física y Astronomía



Introducción y Repaso

Ajuste con `scipy.optimize.curve_fit`

Relación masa–luminosidad

Cierre

Introducción y Repaso

- **Semana 11** se centró en:
 - Ajuste lineal con `np.polyfit`
 - Métricas de calidad: Residuos, R^2 , χ^2_{red}
 - Ponderación por incertidumbres
 - Ejemplo: Ley de Hubble
- **Limitación identificada:** `polyfit` solo funciona con polinomios
- **Conexión** con la sesión actual: necesitamos flexibilidad para ajustar cualquier función

- Comprender `scipy.optimize.curve_fit` para modelos arbitrarios
- **Explorar** la matriz de covarianza y errores de parámetros
- **Aplicar** ajustes no lineales en leyes de potencia
- **Desarrollar** criterio para comparar métodos (log-log vs no lineal)

Ajuste con
`scipy.optimize.curve_fit`

Ajuste con `scipy.optimize.curve_fit` $\ni$ ¿Qué es `curve_fit`?

Definición

`curve_fit` es una función de `scipy.optimize` que ajusta cualquier función arbitraria a datos mediante mínimos cuadrados no lineales.

- **Diferencia clave con `polyfit`:**

- `polyfit`: limitado a polinomios ($y = a_0 + a_1x + a_2x^2 + \dots$)
- `curve_fit`: **cualquier función** (exponenciales, trigonométricas, físicas personalizadas, etc.)

- **Ventajas principales:**

1. Flexibilidad total en la forma funcional
2. Calcula automáticamente incertidumbres de parámetros
3. Proporciona matriz de covarianza completa

Ajuste con `scipy.optimize.curve_fit` $\ni$ Flexibilidad de `curve_fit`: Ejemplos de funciones

Ejemplos físicos

`curve_fit` puede ajustar modelos como:

- Decaimiento exponencial: $N(t) = N_0 e^{-\lambda t}$ (radiactividad)
- Oscilador amortiguado: $x(t) = A e^{-\gamma t} \cos(\omega t + \phi)$
- Ley de potencia: $L = k M^\alpha$ (masa-luminosidad)
- Gaussiana: $f(x) = A e^{-(x-\mu)^2/(2\sigma^2)}$ (perfiles espectrales)
- Planck: $B(\lambda, T) = \frac{2hc^2}{\lambda^5} \frac{1}{e^{hc/(\lambda k_B T)} - 1}$
- Y cualquier combinación o modelo personalizado!

Clave

Solo necesitas definir tu función en Python y `curve_fit` encuentra los parámetros óptimos.

Ajuste con `scipy.optimize.curve_fit` $\ni$ Estructura básica de `curve_fit`

```
1 from scipy.optimize import curve_fit
2
3 # Paso 1: Definir tu función (modelo físico)
4 def mi_modelo(x, param1, param2, ...):
5     """Función que describe tu fenómeno físico"""
6     return ... # fórmula que depende de x y parámetros
7
8 # Paso 2: Llamar a curve_fit
9 popt, pcov = curve_fit(
10     mi_modelo,          # función a ajustar
11     x_datos,           # variable independiente
12     y_datos,           # datos observados
13     p0=[...],          # valores iniciales (opcional)
14     sigma=sigma_datos, # incertidumbres (opcional)
15     absolute_sigma=True # usar sigma como absolutos
16 )
17
18 # Paso 3: Extraer resultados
19 params_optimos = popt      # parámetros ajustados
20 incertidumbres = np.sqrt(np.diag(pcov)) # errores
```

Ajuste con `scipy.optimize.curve_fit` $\ni$ `curve_fit` en un modelo lineal (ejemplo simple)

```
1 from scipy.optimize import curve_fit
2 import numpy as np
3
4 # Definir modelo lineal:  $y = m \cdot x + b$ 
5 def modelo_lineal(x, m, b):
6     return m * x + b
7
8 # Datos del ejemplo de Hubble (del código anterior)
9 # d = distancias, v_obs = velocidades observadas
10 # sigma_v = incertidumbre en velocidades
11
12 # Ajustar el modelo a los datos
13 popt, pcov = curve_fit(
14     modelo_lineal,          # nuestra función
15     d,                      # x: distancias
16     v_obs,                  # y: velocidades observadas
17     sigma=sigma_v,          # incertidumbres
18     absolute_sigma=True     # tratar sigma como valores reales
19 )
20
21 m_cf, b_cf = popt # extraer parámetros optimizados
22 print(f"Pendiente: m = {m_cf:.2f} km/s/Mpc")
23 print(f"Intercepto: b = {b_cf:.2f} km/s")
```

Ajuste con `scipy.optimize.curve_fit` $\ni$ Matriz de covarianza: ¿Qué es `pcov`?

Definición

`pcov` es la matriz de covarianza de los parámetros ajustados. Contiene información sobre las incertidumbres y correlaciones entre parámetros.

- Para 2 parámetros (ej: m, b):

$$\text{pcov} = \begin{pmatrix} \sigma_m^2 & \text{cov}(m, b) \\ \text{cov}(m, b) & \sigma_b^2 \end{pmatrix}$$

- Diagonal (`np.diag(pcov)`): varianzas σ_i^2 de cada parámetro
- Fuera de diagonal: covarianzas entre parámetros
- Incertidumbres: $\sigma_i = \sqrt{\text{pcov}[i, i]}$

Importante

Si `absolute_sigma=True`, las incertidumbres son reales. Si es `False` (por defecto), son relativas y deben escalarse.

Ajuste con `scipy.optimize.curve_fit` $\ni$ Extrayendo incertidumbres de `pcov`

```
1 # pcov es una matriz 2x2 para el modelo lineal (m, b)
2 print("Matriz de covarianza:")
3 print(pcov)
4 # Ejemplo de salida:
5 # [[0.0234 -0.1102]
6 #  [-0.1102  1.4567]]
7
8 # Extraer incertidumbres (errores estándar)
9 perr = np.sqrt(np.diag(pcov))
10 sigma_m = perr[0] # incertidumbre de la pendiente
11 sigma_b = perr[1] # incertidumbre del intercepto
12
13 print(f"\nParámetros con incertidumbres:")
14 print(f"m = {m_cf:.3f} ± {sigma_m:.3f} km/s/Mpc")
15 print(f"b = {b_cf:.3f} ± {sigma_b:.3f} km/s")
16
17 # Interpretación física para Hubble:
18 # H0 = 70.5 ± 2.3 km/s/Mpc significa que con ~68% de confianza,
19 # el valor real está entre 68.2 y 72.8 km/s/Mpc
```

Ajuste con `scipy.optimize.curve_fit` $\ni$ ¿Qué representa `sigma` (σ)?

Definición simple

`sigma` (σ_i) es la **incertidumbre** o **error** asociado a cada medición. Nos dice qué tan confiable es cada dato.

- Interpretación física:
 - σ pequeño: medición precisa, alta confianza
 - σ grande: medición imprecisa, baja confianza
- ¿De dónde viene σ ?
 - Especificaciones del instrumento de medición
 - Repetición de mediciones (desviación estándar)
 - Análisis de fuentes de error experimental
- **Ejemplo concreto:** Si medimos una temperatura como $T = 25.3 \pm 0.5$ C, entonces $\sigma = 0.5$ C

Ajuste con `scipy.optimize.curve_fit` $\ni$ Importancia de σ en los ajustes

- En **`curve_fit`**: el parámetro **`sigma`** permite ponderar los datos
 - Puntos con σ pequeño $\rightarrow$ mayor peso en el ajuste
 - Puntos con σ grande $\rightarrow$ menor peso en el ajuste
- En χ^2 **reducido**: σ es esencial para evaluar consistencia

$$\chi_{\text{red}}^2 = \frac{1}{\nu} \sum_{i=1}^N \frac{(y_i - y_{\text{fit},i})^2}{\sigma_i^2}$$

- **Resultado**: Ajustes más realistas que reflejan la calidad de cada medición

Consejo práctico

Si no conoces σ_i individual, puedes usar el mismo valor para todos los puntos (incertidumbre uniforme).

Ajuste con `scipy.optimize.curve_fit` $\ni$ Visualización completa con `curve_fit`

```
1 import matplotlib.pyplot as plt
2
3 # Graficar datos con barras de error
4 plt.errorbar(d, v_obs, yerr=sigma_v, fmt='o', alpha=0.6,
5             label='Datos observados', capsize=3)
6
7 # Generar línea de ajuste suave
8 d_fit = np.linspace(d.min(), d.max(), 100)
9 v_fit = modelo_lineal(d_fit, *popt)
10 plt.plot(d_fit, v_fit, 'r-', lw=2,
11         label=f'Ajuste:  $H_0 = \{m\_cf:.1f\} \pm \{perr[0]:.1f\}$  km/s/Mpc')
12
13 plt.xlabel('Distancia (Mpc)')
14 plt.ylabel('Velocidad (km/s)')
15 plt.title('Ley de Hubble con curve_fit')
16 plt.legend()
17 plt.grid(alpha=0.3)
18 plt.show()
19
20 print(f"\nIncertidumbre relativa:  $\{ (perr[0]/m\_cf) * 100 : .1f \} \%$ ")
```

Ajuste con `scipy.optimize.curve_fit` $\ni$ Comparación: `curve_fit` vs `polyfit`

Característica	<code>polyfit</code>	<code>curve_fit</code>
Tipos de función	Solo polinomios	Cualquiera
Incertidumbres (σ_i)	Pesos w	Directo
Errores de parámetros	No (manual)	Sí (automático)
Matriz covarianza	No	Sí
Valores iniciales	No necesita	A veces necesario
Velocidad	Rápido	Más lento
Simplicidad	Muy simple	Moderada

Cuándo usar cada uno

- **`polyfit`**: ajustes polinomiales simples, exploración rápida
- **`curve_fit`**: modelos físicos complejos, necesitas errores rigurosos

Relación masa–luminosidad

Relación masa–luminosidad $\ni$ Potencia: $L = k M^\alpha$. Enfoque log–log

```
1 # Simular estrellas
2 rng = np.random.default_rng(1)
3 alpha_true = 3.5
4 k_true = 1.0
5 M = np.linspace(0.5, 10, 40)
6 L_true = k_true * M**alpha_true
7 sigma_rel = 0.10
8 L_obs = L_true * (1 + rng.normal(0, sigma_rel,
  ↳ size=M.size))
9
10 # Transformación log10
11 X = np.log10(M)
12 Y = np.log10(L_obs)
13 a, b = np.polyfit(X, Y, 1) # Y = a X + b
```

```
16 alpha_hat = a
17 k_hat = 10**b
18 print(f"alpha≈{alpha_hat:.2f}, k≈{k_hat:.2f}")
19
20 import matplotlib.pyplot as plt
21 plt.scatter(X, Y, s=20)
22 plt.plot(X, a*X + b, 'r')
23 plt.xlabel('log M'); plt.ylabel('log L')
24 plt.title('Masa–Luminosidad (log–log)')
25 plt.grid(alpha=0.3); plt.show()
```

Discusión: la linealización facilita el ajuste de leyes de potencia.

Relación masa–luminosidad $\ni$ ¿Qué hace este código (log–log)?

- Simula M y L con una ley de potencia y ruido multiplicativo (más realista en astrofísica).
- Transforma a $\log$ para linealizar: $\log L = \alpha \log M + \log k$.
- Ajusta con **polyfit** en el espacio logarítmico y recupera α y k .
- Discute diferencias con el ajuste no lineal en el espacio original.

Relación masa–luminosidad $\ni$ Potencia directa con `curve_fit` (no lineal)

```
1 from scipy.optimize import curve_fit
2
3 def power_law(M, k, alpha):
4     return k * M**alpha
5
6 popt, pcov = curve_fit(power_law, M, L_obs, p0=(1.0, 3.0))
7 k_cf, alpha_cf = popt
8 perr = np.sqrt(np.diag(pcov))
9 print(f"k={k_cf:.2f}±{perr[0]:.2f}, alpha={alpha_cf:.2f}±{perr[1]:.2f}")
10 # Graficar resultados
11 plt.scatter(M, L_obs, s=20, label='estrellas')
12 M_fit = np.linspace(0.5, 10, 100)
13 plt.plot(M_fit, power_law(M_fit, *popt), 'r', label='fit potencia')
14
```

Comparación: log–log vs ajuste no lineal; considerar sesgos con ruido multiplicativo.

- α captura cómo cambia la **producción de energía** con la masa (tasa de fusión y estructura interna).
- Para estrellas de la secuencia principal, típicamente $\alpha \approx 3-4$ en ciertos rangos de masa.
- **Escalas y unidades:** cuidado con trabajar en unidades consistentes; la forma funcional es adimensional si se normaliza.
- **Log–log vs no lineal:** la transformación logarítmica linealiza pero cambia la distribución de errores; el ajuste no lineal trabaja en el espacio original.

Cierre

- Vimos `curve_fit` para ajuste de funciones arbitrarias.
- Extracción de incertidumbres desde la matriz de covarianza.
- Ejemplo: Relación masa-luminosidad (log-log vs no lineal).
- Próxima sesión (P2): Práctica con interpolación, selección de modelos y aplicaciones avanzadas.

- SciPy `curve_fit`
- NumPy `polyfit`
- Matplotlib para visualización

¡Buen trabajo hasta la próxima sesión!

- Practicar con datasets propios (p. ej., decaimientos, oscilaciones, relaciones de escala).
- Traer dudas para profundizar en P2.